

Object-Oriented Programming (OOP) in C# - Basic Concepts & 5 Pillars

Basic Concepts:

Class:

Blueprint for creating objects. Contains fields (variables), methods, properties, and events.

Example:

```
class Car {  
    public string Model;  
    public void Drive() { Console.WriteLine("Driving..."); }  
}
```

Object:

Instance of a class.

Example:

```
Car car1 = new Car();  
car1.Model = "Toyota";  
car1.Drive();
```

Fields & Properties:

Fields store data; properties provide controlled access.

```
private int _speed;  
public int Speed { get { return _speed; } set { _speed = value; } }
```

Methods:

Define behavior.

```
public void StartEngine() { Console.WriteLine("Engine started"); }
```

Constructors:

Special method to initialize objects.

```
public Car(string model) { Model = model; }
```

5 Pillars of OOP (C#):

Encapsulation

Hiding internal data and providing access through methods/properties.

Achieved using: Access Modifiers (public, private, protected).

Example:

```
class Person {  
    private string name;  
    public string Name { get { return name; } set { name = value; } }  
}
```

Abstraction

Hiding complex implementation and showing only essential features.

Achieved using: Abstract classes and Interfaces.

Example:

```
abstract class Shape {  
    public abstract void Draw();  
}  
  
class Circle : Shape {  
    public override void Draw() { Console.WriteLine("Drawing Circle"); }  
}
```

Inheritance

Acquiring properties and behaviors from another class.

Base class (parent) and Derived class (child).

Example:

```
class Animal {  
    public void Eat() { Console.WriteLine("Eating"); }  
}  
  
class Dog : Animal {  
    public void Bark() { Console.WriteLine("Barking"); }  
}
```

Polymorphism

Compile-time (Method Overloading) and Runtime (Method Overriding) polymorphism.

Allows: Same method name with different implementations.

Example (Overriding):

```
class Animal {  
    public virtual void Sound() { Console.WriteLine("Animal sound"); }  
}  
  
class Dog : Animal {  
    public override void Sound() { Console.WriteLine("Bark"); }  
}
```

Association (including Aggregation & Composition)

Association: Relationship between classes.

Aggregation: Weak association (independent life).

Composition: Strong association (dependent life).

Example (Aggregation):

```
class Engine { }  
class Car {  
    public Engine engine;  
    public Car(Engine eng) { engine = eng; }  
}
```

Example (Composition):

```
class House {  
    private Room room = new Room(); // Room cannot exist without House.  
}
```

Summary of Pillars:

Pillar	Purpose	Tools Used
Encapsulation	Hide data, expose essential parts	Access Modifiers, Properties
Abstraction	Hide complexity, show essential behavior	Abstract Classes, Interfaces
Inheritance	Reuse and extend functionality	: keyword, base/derived class
Polymorphism	Multiple forms of same method	Overriding, Overloading
Association	Relationships between classes	Aggregation, Composition

Data Structures and Algorithms (DSA) - Basic Concepts, Sorting, and Searching

Basic Concepts:

Data Structure:

A way to store and organize data efficiently. Examples: Arrays, Linked Lists, Stacks, Queues, Trees, Graphs.

Algorithm:

A step-by-step procedure to solve a problem or perform a task.

Time Complexity:

Amount of time an algorithm takes to complete as a function of input size (Big O notation).

Space Complexity:

Amount of memory an algorithm uses as a function of input size.

Recursion:

A function that calls itself to solve smaller instances of a problem.

Sorting Algorithms:

1. Bubble Sort:

Repeatedly swaps adjacent elements if they are in the wrong order.

Time Complexity: $O(n^2)$

Example:

```
for (int i = 0; i < n-1; i++)
    for (int j = 0; j < n-i-1; j++)
        if (arr[j] > arr[j+1])
            swap(arr[j], arr[j+1]);
```

2. Selection Sort:

Selects the minimum element from the unsorted part and swaps it with the beginning.

Time Complexity: $O(n^2)$

Example:

```
for (int i = 0; i < n-1; i++)
    for (int j = i+1; j < n; j++)
        if (arr[j] < arr[min_idx])
            min_idx = j;
```

3. Insertion Sort:

Builds the final sorted array one item at a time.

Time Complexity: $O(n^2)$

Example:

```
for (int i = 1; i < n; i++)
    key = arr[i];
```

```
j = i - 1;
while (j >= 0 && arr[j] > key)
    arr[j + 1] = arr[j--];
arr[j + 1] = key;
```

4. Merge Sort:

Divide and Conquer algorithm that divides array into halves, sorts, and merges them.

Time Complexity: $O(n \log n)$

Example:

Divide array, sort each half, merge sorted halves.

5. Quick Sort:

Divide and Conquer algorithm. Picks a pivot and partitions array.

Time Complexity: $O(n \log n)$

Example:

Partition the array, recursively sort partitions.

6. Heap Sort:

Builds a Heap and repeatedly extracts maximum element.

Time Complexity: $O(n \log n)$

Example:

Build max heap, swap root with last, heapify.

Searching Algorithms:

1. Linear Search:

Searches each element until the target is found.

Time Complexity: $O(n)$

Example:

```
for (int i = 0; i < n; i++)
    if (arr[i] == target)
        return i;
```

2. Binary Search:

Efficient search on sorted arrays by dividing search space.

Time Complexity: $O(\log n)$

Example:

```
while (low <= high)
    mid = (low + high)/2;
    if (arr[mid] == target)
        return mid;
    else if (arr[mid] < target)
        low = mid + 1;
    else
        high = mid - 1;
```

3. Jump Search:

Search by jumping ahead by fixed steps and then linear search.

Time Complexity: $O(\sqrt{n})$

Useful for sorted arrays.

4. Interpolation Search:

Improved variant of Binary Search for uniformly distributed data.

Time Complexity: $O(\log \log n)$ best case.

5. Exponential Search:

Finds range where element lies and performs Binary Search.

Time Complexity: $O(\log n)$

Summary of Sorting Algorithms:

Algorithm	Time Complexity	Type
Bubble Sort	$O(n^2)$	Comparison-based
Selection Sort	$O(n^2)$	Comparison-based
Insertion Sort	$O(n^2)$	Comparison-based
Merge Sort	$O(n \log n)$	Divide and Conquer
Quick Sort	$O(n \log n)$	Divide and Conquer
Heap Sort	$O(n \log n)$	Heap-based

Summary of Searching Algorithms:

Algorithm	Time Complexity	Type
Linear Search	$O(n)$	Sequential
Binary Search	$O(\log n)$	Divide and Conquer
Jump Search	$O(\sqrt{n})$	Block-based
Interpolation Search	$O(\log \log n)$	Probing-based
Exponential Search	$O(\log n)$	Range Expansion

HTML, CSS, and JavaScript Basic Concepts

HTML (HyperText Markup Language)

1. Introduction to HTML

- HTML is the standard markup language used to create web pages.
- It defines the structure of web content.
- HTML elements are represented by tags.

2. Basic Structure of an HTML Document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

3. Common HTML Elements

- **Headings:** <h1> to <h6>
- **Paragraph:** <p>
- **Links:**
- **Images:**
- **Lists:**
 - **Ordered:** Item
 - **Unordered:** Item
- **Forms:** <form>, <input>, <button>
- **Tables:** <table>, <tr>, <td>

CSS (Cascading Style Sheets)

1. Introduction to CSS

- CSS is used to style HTML elements.
- It enhances the visual presentation of web pages.
- Can be applied via Inline, Internal, and External CSS.

2. Basic CSS Syntax

```
selector {
  property: value;
}
```

Example:

```
body {
  background-color: lightblue;
}
```

3. CSS Selectors

- **Universal Selector** (*)
- **Element Selector** (element)
- **Class Selector** (.)
- **ID Selector** (#)

- **Grouping Selector (``)**
- **Attribute Selectors (``)**

4. Box Model

- Consists of margin, border, padding, and content.
- Determines element dimensions and spacing.

5. Flexbox

- Used for flexible and responsive layouts.
- Parent container (`display: flex;`) and child properties control alignment.

Flexbox Properties

- `display: flex;`
- `justify-content`: (center, space-between, space-around)
- `align-items`: (flex-start, center, flex-end)
- `flex-direction`: (row, column)

6. CSS Grid

- Used to design complex, responsive layouts.
- Grid container: `display: grid;`
- Define columns/rows: `grid-template-columns / grid-template-rows`

Example:

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
```

JavaScript (JS)

1. Introduction to JavaScript

- JavaScript is a programming language that adds interactivity to web pages.
- Runs in the browser and manipulates the DOM (Document Object Model).

2. Basic JavaScript Syntax

```
console.log("Hello World");
```

3. Variables and Data Types

- `var`, `let`, `const` for variable declarations.
- Data types: Number, String, Boolean, Object, Array.

Example:

```
let name = "John";
const age = 25;
```

4. Functions in JavaScript

```
function greet(name) {
  return "Hello, " + name;
}
console.log(greet("Alice"));
```

5. Event Handling

```
document.getElementById("btn").addEventListener("click", function() {
  alert("Button Clicked!");
});
```



```
});
```

6. DOM Manipulation

- **Select elements:** `document.getElementById()`, `document.querySelector()`
- **Modify content:** `.innerHTML`, `.textContent`
- **Modify styles:** `.style.color = "red";`

7. Loops and Conditional Statements

- `if`, `else`, `for`, `while`

Example:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

8. JavaScript ES6 Features

- `let` and `const`
- **Arrow functions:** `const sum = (a, b) => a + b;`
- **Template literals:** ``Hello, ${name}``
- **Destructuring:** `const {name, age} = person;`
- **Spread Operator:** `const newArr = [...arr1, ...arr2];`

End of Notes